

C语言阶段

知识体系

单元三：函数与重构

能力考评

1、写出以下代码段的输出结果

```
void swap(int m,int n)
{
    int temp;
    temp = m;m = n;n = temp;
}
main()
{
    int m = 7,n = 5;
    swap(m,n);
    printf("m = %d,n = %d\n",m,n);
}
```

只有函数里的变量进行交换

输出结果： m=7,n=5

2、请写出下列代码的输出内容 19

```
#define add(a,b) a+b
int main()
{
    printf("%d\n", 5 * add(3, 4));
    return 0;
}
```

宏函数

3、x = 5858,下面函数的返回值为 8

```
int foo(int x)
{
    int c = 0;
    while(0 != x)
    {
        ++c;
        x = x&(x-1);
    }
    return c;
}
```

5858二进制是1011000110110
第一次循环后c变为1, x变为1011000110110&1011000110110-1=1011000110100
然后每次循环后都会将x最低位清零并且将c+1
因为5858的二进制里有8个1, 所以foo返回值为8

4、在64位机器下的输出为 80

```
int(*p[10])(int);
printf("%d", sizeof(p));
```

5、程序中如果有如下宏定义，#define RECTANGLE_AREA(a, b) a*b

如果 int i = 3, j = 4; 那么

请问：RECTANGLE_AREA(i + 5, i + j);的结果是 22

6、为“五常：仁义礼智信”创建一个名为：五常的枚举类型： enum 五常{仁，义，礼，智，信};

7、假设下面的代码申请128字节内存都成功，那么下面代码的运行结果是什么？哪一个分支会导致内存泄露，为什么？

```
#include <stdio.h>
#include <malloc.h>
void modifyPionter(char* input)
{
    char* tmp = malloc(128);
    input = tmp;
    return;
}
void modifyPionter2(char** input)    二级指针
{
    char* tmp = malloc(128);
    *input = tmp;                    以形参返回方式将数据带回
    return;
}
char* modifyPionter3(void)
{
    char* tmp = malloc(128);    output3直接通过modifyPionter3函数返回的地址被赋值
    return tmp;
}
int main()
{
    char* output1 = NULL;
    char* output2 = NULL;
    char* output3 = NULL;
    modifyPionter(output1);
    modifyPionter2(&output2);
    output3 = modifyPionter3();
    if (output1)
    {
        printf("output1 Branch\n");
        free(output1);
    }
    if (output2)
    {
        printf("output2 Branch\n");
        free(output2);
    }
    if (output3)
    {
        printf("output3 Branch\n");
        free(output3);
    }
    return 0;
}
```

会导致内存泄露的分支 modifyPionter函数调用.

原因

由于modifyPointer没有返回值output1指向的内存不再有效;

8、下面代码，i和j的值分别是什么 10，19

```
static int j;    静态全局作用域限于本源代码
int k = 0;       全局变量作用域整个项目内
void fun1()
{
    static int i;
    i++;
}               每次循环调用fun1 ()，i的值都会增加1，因为i是静态局部变量
               在第一次调用fun10时，i初始化为0，然后每次调用都会增加1
void fun2()
{
    j = 0;       每次循环都会调用fun2()，j的值都会重置为0，然后增加到1，因此j始终为1
    j++;
}
int main()
{
    for ( ; k<10 ;k++ )
    {
        fun1();
        fun2();
    }
    fun1();
    fun2();
    return 0;
}
```

for循环后fun1 () 和fun2() 再次被调用
fun1()调用后i值为10
fun2()调用后j被重置为0然后增加到1

9、下面代码在x86下，各sizeof的计算值

```
int b[100];
void fun(int b[100])
{
    sizeof(b); //的值 4
}
int main()
{
    int* p = NULL;
    sizeof(p); //的值 4
    sizeof(*p); //的值 4
    int a[100];
    sizeof(a); //的值 400
    sizeof(a[100]); //的值 4
    sizeof(int[100]); //的值 400
    sizeof(&a); //的值 4
    sizeof(&a[0]); //的值 4
    return 0;
}
```

10、使用typedef对如下要求的指针类型进行命名：

A：为：long long 类型的指针创建一个名为P_LL的指针类型: typedef long long*P_LL;

B：为:能指向short[5]数组的指针创建一个名为P_S5的数组指针类型:_____。
`typedef short (*P_S5)[5];`

C：为:能指向float[3][5]数组的指针创建一个名为P_F35的数组指针类型:_____。
`typedef float(*P_F35)[3][5];`

D：为:能指向double sqrt(double x)函数的指针创建一个名为P_S5的函数指针类型:_____。
`typedef double(*P_S5)(double)`